ARM Cortex-M Development Environment Setup (Ubuntu 24.04)

Step 1: Update the System

```
sudo apt update
sudo apt upgrade -y
```

Step 2: Install Git

```
sudo apt install git -y
```

Verify:

```
git --version
```

Expected output:

```
git version 2.x.x
```

Step 3: Install ARM GNU Toolchain

```
sudo apt install gcc-arm-none-eabi -y
```

Verify:

```
arm-none-eabi-gcc --version
```

Expected output:

```
arm-none-eabi-gcc (15:13.2.rel1-2) 13.2.1
```

Step 4: Verify ARM Utilities

```
arm-none-eabi-objcopy --version
```

```
arm-none-eabi-objdump --version
```

```
arm-none-eabi-size --version
```

```
arm-none-eabi-nm --version
```

```
arm-none-eabi-ar --version
```

```
arm-none-eabi-gdb --version
```

All commands should display version information.

Step 5: Install Multi-Architecture GDB

```
sudo apt install gdb-multiarch -y
```

Verify:

```
gdb-multiarch --version
```

Step 6: Install Build Tools

```
sudo apt install build-essential -y
```

Verify:

```
make --version
```

```
gcc --version
```

Step 7: Install CMake

```
sudo apt install cmake -y
```

Verify:

```
cmake --version
```

Step 8: Install Ninja

```
sudo apt install ninja-build -y
```

Verify:

```
ninja --version
```

Step 9: Install OpenOCD

```
sudo apt install openocd -y
```

Verify:

```
openocd --version
```

Step 10: Install USB Utilities

```
sudo apt install usbutils -y
```

Verify:

```
lsusb
```

Step 11: Install Serial Communication Tools

```
sudo apt install minicom screen picocom -y
```

Verify:

```
minicom --version
```

Step 12: Install Tree Utility

```
sudo apt install tree -y
```

Verify:

```
tree --version
```

Step 13: Install Python

```
sudo apt install python3 python3-pip python3-venv -y
```

Verify:

```
python3 --version
```

Step 14: Install Device Firmware Upgrade Utility

```
sudo apt install dfu-util -y
```

Verify:

```
dfu-util --version
```

Step 15: Install Useful Development Libraries

```
sudo apt install \
libusb-1.0-0-dev \
pkg-config \
libncurses-dev \
libtool \
autoconf \
automake \
flex \
bison \
gperf \
wget \
curl \
unzip \
zip -y
```

Step 16: Add User to the Dialout Group

This allows access to serial ports without using `sudo`.

```
sudo usermod -aG dialout $USER
```

After running this command:

- Log out.
- Log back in.

Verify:

```
groups
```

You should see:

```
dialout
```

Step 17: Connect the Vaaman Board

Run:

```
lsusb
```

Expected:

```
1d50:6140
```

Check the serial port:

```
ls /dev/ttyACM*
```

Expected:

```
/dev/ttyACM0
```

Step 18: Test USB Communication

```
sudo dmesg | tail -20
```

Expected output:

```
cdc_acm
ttyACM0
```

Step 19: Create a Workspace

```
mkdir -p ~/ARM_Projects
cd ~/ARM_Projects
```

Step 20: Clone the Reference Repository

```
git clone https://github.com/pradyumnasv9/digital-design.git
```

Navigate to the ARM folder:

```
cd digital-design/vaman/arm
```

View the directory structure:

```
tree -L 2
```

Final Verification Checklist

Run these commands:

```
arm-none-eabi-gcc --version
```

```
arm-none-eabi-gdb --version
```

```
openocd --version
```

```
cmake --version
```

```
make --version
```

```
git --version
```

```
python3 --version
```

```
lsusb
```

```
ls /dev/ttyACM*
```

All of these should complete successfully.

Expected Development Environment

After completing the installation, the laptop will be ready with:

- ✅ ARM GNU GCC Toolchain (`arm-none-eabi-gcc`)
- ✅ ARM GDB Debugger
- ✅ OpenOCD
- ✅ CMake
- ✅ Make
- ✅ Ninja Build
- ✅ Git
- ✅ Python 3
- ✅ USB utilities
- ✅ Serial communication tools
- ✅ DFU utility
- ✅ Vaaman board detection (`/dev/ttyACM0`)

This provides a solid foundation for developing and debugging **bare-metal ARM Cortex-M** applications. Once the board support package (startup code, linker script, and flashing mechanism) is available, you can begin building applications such as LED blinking, GPIO control, UART communication, timers, interrupts, and later ARM–FPGA communication on the Vaaman board.